ARCHITECTURE REVIEW · PRE-IMPLEMENTATION

SADINhub ML Rebuild Review

A read of the five planning documents against the live database before any Task 0 code is written. The headline result: the leakage problem that blocks the whole plan is measurably zero at daily grain, and the finding that justifies the weather workstream reverses sign when the project's own long-gap caveat is applied.

Scope Tasks 0–4, no code written **Verified against** PostgreSQL 18.4, read-only **Data** 2024-12-09 → 2026-07-11

Findings 125 catalogued

Everything below was checked against the live `sadinhub` database over a read-only connection. Every documented volume reconciles exactly — 4,535,918 snapshot rows, 4,260,849 validated rows, 2,020,697.5 kWh, 29 of 31 meters. The pipeline document is accurate where it counts. The problems are concentrated in the three newer ML documents, and two of them change what should be built first.

23	0.00%	−11%	9
readings in 4.26M touched by forward-looking cleaning	replay divergence from Gold at end-of-day as_of	true weekend vs weekday (docs claim higher)	of 15 column names in the Model 1 query do not exist

Section 01

The two findings that change the plan

FINDING A · TASK 0 DOES NOT NEED TO BLOCK TASKS 2 AND 3

The plan calls the leakage-safe replay "the single highest-leverage task in the whole plan — every downstream metric depends on it," and the founder briefing describes it as a week of work that must land before any forecast number can be trusted. Measured against the live table, the forward-looking cleaning rules touch **23 readings out of 4,260,849**.

Those 23 fall on **two Saudi dates only** — 2025-02-04 (22 meters) and 2026-06-27 (one meter, ZN.3-PB.1) — and affect 23 meter-days out of 14,753, or 0.156%. Decisively: **every one of the 23 has its successor inside the same Saudi day, and none is the last reading of its day.** So with `as_of` set to end-of-day, a point-in-time replay reproduces Gold *exactly* for 100% of historical rows. The Task 0 deliverable `pct_rows_replay_differs_from_retrospective` will read 0.0000%.

Build Task 0 — but build it as a *proof* and a forward guard, not a gate. It takes days, not a week, and Tasks 2 and 3 can start immediately against current Gold without any metric being wrong.

The rule still earns its place, because it is rare but severe when it fires. The 2026-06-27 event on ZN.3-PB.1 ran `35272.4 → 94894 → 35272.8` . A naive real-time reading posts **+59,621.6 kWh** on a meter that normally does about 100 kWh a day. The quality screen would not catch it, because the screen tests coverage and gaps, not magnitude. In future data a spike landing on a day's *final* reading would be genuinely unresolvable — that is the case Task 0 exists to handle, and it is worth stating that way rather than as a correction to history.

FINDING B · THE WEEKEND ANOMALY IS AN ARTIFACT AND REVERSES

The founder briefing offers "weekend usage is *higher* than weekday usage, which is unusual for most facilities" as evidence that a hidden schedule is driving load, and Plan §5 names it as the reason to acquire a shift calendar. Recomputed from `gold.energy_consumption_validated` on Saudi dates with Friday and Saturday as the weekend:

BASIS	WEEKDAY (SUN–THU)	WEEKEND (FRI/SAT)	READS AS
All intervals, incl. `LONG_GAP`	128.3 kWh	158.7 kWh	WEEKEND +24%
`NORMAL_INTERVAL` only	116.6 kWh	104.0 kWh	WEEKEND −11%
Clean days only (no gap touched)	117.7 kWh	104.7 kWh	WEEKEND −11%

The sign reverses. Long-gap consumption is assigned in full to the first reading after telemetry resumes, so a handful of gap recoveries landing on Fridays carries the entire effect — the pipeline document warns about exactly this in §6.7 and §10, and the EDA headline did not apply its own caveat. A commercial facility in Riyadh using **11% less** at the weekend is unremarkable and needs no explanation.

This does not invalidate the weather workstream — cooling load in Riyadh is a real and probably strong signal, and `gold.zone` carries the coordinates (24.69751, 46.66536, DIOM Facility) so sourcing it is straightforward. It removes the *occupancy/shift calendar* justification, and it removes the claim that something unexplained is happening. Worth correcting in the briefing before it is read as a finding.

> ONE CORRECTION TO THE RECORD
>
> The models were never misled by this. The daily quality screen already excludes long-gap-touched meter-days, so the training data sits on the correct side of the table. Only the written narrative was wrong.

Section 02

How data actually flows, Bronze to ML

The pipeline document's description is accurate; what follows adds only what I verified live and what the ML documents get wrong about the handoff.

Bronze — untouched source

`bronze.telemetries` holds roughly 109 million long-format rows, one measurement per row, keyed by `(device_id, key_id, timestamp)` with epoch-millisecond time and the value in `dbl`. A reading packet is a unique `(device_id, timestamp)` pair carrying about 24 populated electrical keys. `bronze.energy_meters` and `bronze.telemetries_dictionaries` supply identity and the key-to-name map. Nothing is trimmed, interpolated or overwritten here.

Silver — where every business rule lives

Four stages, in order. `fact_energy_readings_staging` pivots the 32 electrical keys into one wide row per meter-timestamp and derives UTC, Saudi and Saudi-date columns plus packet diagnostics. `fact_energy_readings_flagged` applies the `classify_*` functions to produce a status per measurement. `fact_energy_readings_snapshot` materialises the join of the two — **4,535,918 rows, 31 meters**, primary key `(energy_meter_key, reading_timestamp_utc)`. Then `fact_energy_consumption_validated` turns the cumulative register into per-interval kWh — **4,260,849 rows, 29 meters**, Zone 13 excluded.

Two classifier details the ML documents need and do not state. The current/power agreement rule fires only below 15% of the meter's breaker rating and uses literal zero comparisons, which is what produces `Major Issue` and `Major Problem`. And `overall_status` is the worst of exactly twelve inputs that **deliberately exclude the three current statuses** — so any anomaly work must read `status_amp1/2/3` directly or it will silently miss the entire current-mismatch family, which is the evidence behind ZN.20-PB.1.

Gold — governed views at native grain

Ten views plus one physical gap table. Two matter here. `gold.energy_raw` is a pass-through of the snapshot: 52 columns, 31 meters, every status. `gold.energy_consumption_validated` is a pass-through of validated consumption: 15 columns, 29 meters, and it carries `meter_label`, both Saudi timestamp columns, `consumption_cleaning_status`, `interval_quality` and `gap_minutes`.

> THE ASYMMETRY EVERY ML TASK TRIPS OVER
>
> `gold.energy_raw` has **no `meter_label` and no Saudi timestamp columns at all** — only `reading_timestamp_utc`. `gold.energy_consumption_validated` has all three. So Model 1 and Model 3, which read the raw fact, must join `gold.meter` for labels and derive the Saudi date themselves as UTC+3 (Asia/Riyadh, no DST). Both documents assume these columns are present.

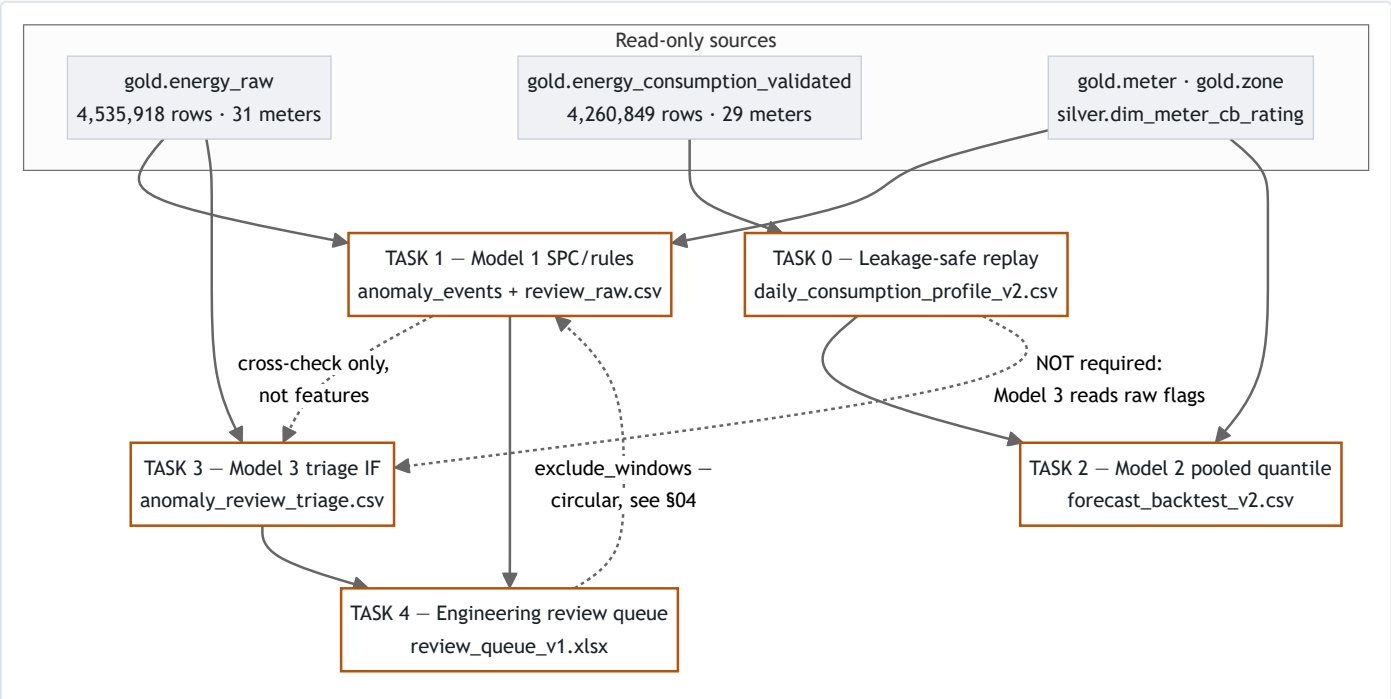
The ML branch

Read-only Postgres in, local files out, nothing written back. The interval→day handoff is the part to get right: consumption is assigned to the Saudi-local date of the interval's *ending* reading, duration coverage is split at Saudi midnight, and missing energy is never estimated, zero-filled or backfilled. That yields 16,704 calendar meter-days, of which 14,753 carry real readings (the difference is the 1,951 explicitly inserted missing days) and 14,228 pass the quality screen. I confirmed the 14,753 live; the arithmetic in §11.2 holds.

Section 03

Dependency graph, Tasks 0–4

Solid arrows are hard dependencies; dashed are advisory or cross-check only. The graph below is the *corrected* one — the documents assert three dependencies that the evidence does not support, marked in red.



WHERE THE DOCUMENTS DISAGREE WITH THIS GRAPH

- **Task 3 → Task 0 is asserted but unfounded.** The Task Breakdown header says Task 3 "blocks on Task 0 (needs `daily_consumption_profile_v2.csv`)" and the Plan diagram routes Model 3 under the replay. But the only concrete spec, Instructions V2 §3.1, builds Model 3's features purely from `gold.energy_raw` flag counts and never opens the daily profile — while the run order still passes it `--daily-profile`. Model 3 needs Task 0 for nothing. DECISION NEEDED

- **Task 3 → Task 1 is for the cross-check, not features.** The same header says "and ideally Task 1 (for flag-count features)", but Task 1's output is *episodes*, not counts; the counts come from the database. Waiting on Task 1 waits for an artifact that never contains them.
- **Task 4 → Task 1 closes a loop.** Task 1's `exclude_windows` is fed by Task 4's confirmed faults, while Task 4 cannot run until Task 1 has passed its acceptance gate. Section 04 shows why that gate is unpassable as written, which makes the loop a deadlock rather than an iteration.
- **Task 0 → Task 2 is real but should not block.** Per Finding A the replay is provably equivalent on this history, so Task 2 can proceed against current Gold and swap inputs when Task 0 lands.

Section 04

Database objects each task needs

TASK	OBJECTS	COLUMNS THAT MATTER	VERIFIED ISSUE
Task 0 Replay	`gold.energy_consumption_validated` (+ gold.meter for labels)	energy_meter_key, meter_label, reading_timestamp_utc, reading_date_saudi, active_energy_cumulative_kwh, previous_valid_kwh, validated_kwh_consumption, consumption_cleaning_status, interval_quality, gap_minutes	Everything needed is present. Index `idx_consumption_meter_time` supports per-meter replay.
Task 1 Model 1	`gold.energy_raw` `gold.meter` `silver.dim_meter_cb_rating`	active_power_phase_{1,2,3}_kw, active_power_total_kw, power_factor_phase_{1,2,3}, current_phase_{1,2,3}_a, status_amp{1,2,3}, status_pf{1,2,3}, reading_timestamp_utc	`9 OF 15 NAMES IN §1.1 DO NOT EXIST` No meter label, no Saudi date in this view.
Task 2 Model 2	Task 0 CSV `gold.meter`, `gold.zone` `silver.dim_meter_cb_rating`	zone_key → gold.zone.zone_name, cb_max_rating_a, phase_count	`CROSSES THE GOLD-ONLY BOUNDARY` Breaker rating exists in Silver only. `zone_id` and `meter_id` do not exist as named.
Task 3 Model 3	`gold.energy_raw` `gold.meter`	status_amp{1,2,3}, status_pf{1,2,3}, active_power_total_kw, reading_timestamp_utc	`GROUPS BY A SAUDI DATE COLUMN THAT DOES NOT EXIST` Counts need normalising — see below.
Task 4 Review	None — file inputs only	outputs of Tasks 1 and 3	Two incompatible grains, and no shared severity column.

CORRECT COLUMN NAMES FOR THE MODEL 1 PULL

INSTRUCTIONS V2 §1.1 WRITES	ACTUAL COLUMN IN `GOLD.ENERGY_RAW`	
active_power_kw1 / kw2 / kw3	active_power_phase_1_kw / _2_kw / _3_kw	WRONG
power_factor_1 / _2 / _3	power_factor_phase_1 / _2 / _3	WRONG
current_amp1 / 2 / 3	current_phase_1_a / _2_a / _3_a	WRONG
status_amp1 / 2 / 3	status_amp1 / 2 / 3	CORRECT
status_pf1 / 2 / 3	status_pf1 / 2 / 3	CORRECT
— never selected —	active_power_total_kw	NEEDED FOR NEGATIVE-TKW

PULL STRATEGY — STREAM PER METER, NOT ONE GLOBAL SORT

Instructions V2 §1.1 issues a single unordered 4.5M-row `SELECT … ORDER BY energy_meter_key, reading_timestamp_utc`. Measured, that buffers roughly 3.9 GB of Python objects before any DataFrame copy. The snapshot's primary key `(energy_meter_key, reading_timestamp_utc)` is a unique btree that exactly matches per-meter time-ordered access, so looping one meter at a time — 96,471 to 163,947 rows each, 146,320 on average — is an index-ordered scan with no sort and a bounded footprint. Table sizes are 2,287 MB for the snapshot and 1,221 MB for validated; a `count(*)` takes about 5 s and a grouped aggregate 11–35 s, so the 120 s default statement timeout is survivable only if the work is chunked.

Section 05

Ambiguities and contradictions

125 findings were catalogued across seven review dimensions and put through two adversarial passes — one checking the evidence, one arguing the documents' defence. The adversarial pass mattered: it overturned six claims, including one of my own. What follows is the set that changes what gets built, not the full catalogue.

125	32	17	15
findings catalogued	rated critical	survived both adversarial lenses	disputed or overturned on review

Method flaws — these affect whether the numbers mean anything

The per-meter promotion gate selects on the final test window

CRITICAL CONFIRMED

Task Breakdown Task 2 step 7 · Plan §4 · contradicted by Instructions V2 §2.4 and pipeline §11.4

Task Breakdown step 7 keeps the pooled model per meter only if it beats the baseline "on the final test window." Fifteen lines above the gate code, §2.4 says "final_test used once, not for selection," and pipeline §11.4 records that the old experiment deliberately did *not* use the final test to choose the per-meter model. Choosing one of two methods per meter across 29 meters on the test set is selection on the test set; the reported figure is then a maximum over 29 coin flips, not an estimate. Plan §5–6 compound it by re-running the gate after every feature addition.

RESOLUTION — Run the gate on validation (val1+val2 pooled), then report the final test once, unchanged, for whatever the gate chose. This is what the old experiment did, and §11.5 is the cautionary result: validation-based selection scored 12.53% WAPE against the baseline's 12.05% on the same 1,293 observations. Selection did not help; moving it onto the test set will make it look like it did.

The 2pp gate margin is smaller than the noise it is measuring

CRITICAL

Instructions V2 §2.5 `margin_pp: float = 2.0` · Task Breakdown Task 2 step 7

The final test window is 60 days (2026-05-12 to 2026-07-10) across 29 meters = 1,740 possible meter-days, of which §11.5 records 1,344 with complete feature history — about **46 observations per meter**. A per-meter WAPE from 46 observations carries a standard error comfortably above 2 percentage points, so the gate will promote and demote meters on sampling noise. Pipeline §11.4's minimum of 120 training and 20 evaluation observations per meter is also dropped from Task 2 without comment; the evaluation minimum still binds the gate.

RESOLUTION — Either widen the evaluation window, or replace the fixed margin with a paired bootstrap over meter-days and promote only where the interval clears zero. Reinstate the §11.4 minimums explicitly.

Forecast issue time is still undefined

CRITICAL

`All five documents · pipeline §11.7 and §14.2 flag it as blocking`

Pipeline §11.7 lists metric, horizon, forecast issue time, reporting delay, eligible coverage and the per-meter acceptance rule as things that "must be agreed before the target can be assessed," and §14.2 records the acceptance metric as TBD. None of the three ML documents defines any of them. Without an issue time, "target = next_day_kwh" alongside `lag_1` is ambiguous by one day — and a one-day shift is worth several percentage points of WAPE, more than the entire gate margin.

RESOLUTION — Fix it before Task 2 starts: forecast for day D+1 is issued at end of Saudi day D, so `lag_1` is day D. Write it into the spec as a sentence, not an assumption. Note that no document sets an *absolute* bar at all — the gate is purely relative to the naive baseline, so a 0.2pp win counts as success.

Model 1's acceptance test cannot be passed by the detectors it specifies

CRITICAL

`Instructions V2 §1.2–1.4 · Task Breakdown Task 1 steps 2 and 6`

Three problems compound. *Circularity*: §1.2 excludes ZN.16-PB's full history from its own baseline, while §1.4 requires the detector to independently surface ZN.16-PB on all three phases — with no baseline left, it cannot. *Stationarity*: an EWMA against a rolling 30-day baseline detects *change*, and the ZN.16 polarity fault is present across the meter's entire history, so it is not a change point and never crosses a control limit. *Wrong instrument*: EWMA on kW plus CUSUM on power factor cannot produce the current/power-agreement evidence that defines ZN.20-PB.1 — that finding comes from `status_amp*`, which those detectors never read.

RESOLUTION — Add a third detector family for cross-phase and current/power consistency (peer-phase ratio comparison, which is how the Assessment Report found these by hand), and calibrate ZN.16's baseline from peer meters rather than its own history.

CUSUM on power factor at target 0.8 targets the acceptable band, not a fault

HIGH

`Instructions V2 §1.3 cusum_flags(pf, target=0.8) · pipeline §6.5 and §13`

0.8 is the *lower edge of Acceptable*; Critical is below 0.6. More than half of all readings sit below 0.8, and the majority of PF-Critical readings occur at near-zero load. Pipeline §13 is explicit: "Low-load operation can produce low PF, so classifier rates are not proof of a fault. A minimum-load gate remains undecided." Running this as specified floods the review queue with lightly loaded meters.

RESOLUTION — Decide the minimum-load gate now (it is listed as undecided and it blocks Model 1), and set the CUSUM target from each meter's own loaded-hours PF distribution rather than a constant.

Isolation Forest on raw counts learns reporting volume, not electrical behaviour

HIGH

`Instructions V2 §3.1 · Task Breakdown Task 3 step 1`

Readings per meter-day vary with cadence and with missing data — about 286–287 for most meters, about 269 for the Zone 13 pair at their 321 s cadence, and far fewer on partial days. Counting flagged readings per day without dividing by readings reported that day means a day with fewer readings scores as less anomalous for that reason alone. The old run's 53 flags of 1,344 days (3.9%) against a configured 2% contamination is an unexplained gap that this would partly account for.

RESOLUTION — Use rates, not counts, and carry `readings_reported` as an explicit feature so the model can separate the two.

Specification bugs — these fail on first run

The read-only safety assert cannot pass as written

CRITICAL CONFIRMED

Instructions V2 §1 `src/db.py`

psycopg3 defaults to `autocommit=False`, so the first `conn.execute(...)` opens a transaction. `transaction_read_only` is fixed at transaction start, and `SET SESSION CHARACTERISTICS` sets the default for *future* transactions — so it cannot affect the transaction already running, and the assert that follows reads `off`. I reproduced this on the live server. This is the one safety gate the whole engagement rests on.

RESOLUTION — Use the pattern this project already has working in `DATABASE/export_database.py`: pass libpq options at connect time so they apply from the first transaction — `options='-c default_transaction_read_only=on -c default_transaction_isolation=repeatable read -c statement_timeout=…'` — then verify `SHOW transaction_read_only` is `on`. That also delivers the REPEATABLE READ the safety brief requires, which the current code never actually applies.

Neither installed interpreter can run Task 2

CRITICAL

Instructions V2 §1 (`import psycopg`) and §2.3 (`import lightgbm`)

`ML_WORK/.venv` has psycopg 3.3.5 but no lightgbm and no yaml. The base Python 3.12.4 has lightgbm 4.6.0, pandas, scikit-learn and PyYAML but no psycopg. Task 2 needs both, so it runs on neither. `pyarrow` and `openpyxl` are absent entirely, which makes the `anomaly_events.parquet` and `review_queue_v1.xlsx` deliverables unbuildable as specified. No document names an interpreter, an activation step or a dependency set.

RESOLUTION — One environment under `ML_WORK/` with a pinned `requirements.txt`, resolved before Task 0 begins. Exclude it and `outputs/` from OneDrive sync, since the project tree is inside a synced folder.

Both safety gates use bare `assert`

HIGH CONFIRMED

Instructions V2 §1 and §1.1

`python -O` or `PYTHONOPTIMIZE=1` strips assert statements. The read-only verification and the row-count check both disappear silently under a flag someone may reasonably set. A safety control that can be switched off by an optimisation flag is not a control.

RESOLUTION — `raise RuntimeError`. Also drop the hard-coded `4,535,918` assert to a logged warning with a tolerance, or Model 1 fails by design the next time Bronze is refreshed — which pipeline §6.6 says happens as a full rebuild that also drops and recreates `gold.energy_raw`.

The combined review queue has no valid sort key and two incompatible grains

HIGH

Instructions V2 §4 `build_combined_review_queue`

The function concatenates Task 1's output with Task 3's and sorts by `severity`. Task 3's triage CSV has an anomaly score, not a severity — so the concat yields NaN for every Model 3 row and the sort is undefined. Worse, `severity` is required by four separate deliverables and is **never defined in any of the five documents**. The two inputs are also different objects: Model 1 emits episodes (meter × phase × time range), Model 3 emits meter-days. There is also no stable row id to merge engineering's hand-edited labels back.

RESOLUTION — Define severity explicitly as an ordinal with stated inputs, give both sources a shared `event_id`, and normalise Model 3 rows to a one-day episode so a single grain holds.

Cross-document contradictions

SUBJECT	THE DISAGREEMENT	RESOLUTION
Known-finding meters	Plan §4 names **4** candidates (ZN.16, ZN.5, ZN.8.2, ZN.12-PB); Task Breakdown step 6 and Instructions V2 §1.4 name **6**, adding ZN.20-PB.1 and ZN.10-PB.2. Pipeline §13 — the authoritative current-state table — lists **5** and does not include ZN.10-PB.2. Phase assignments for 3 of the 6 also contradict the Assessment Report.	Fix the acceptance set against §13 and the Assessment Report before it becomes a pass/fail gate. ZN.10-PB.2 needs a source or removal.

SUBJECT	THE DISAGREEMENT	RESOLUTION
Meter label format	Documents use `ZN.16-PB` ; the canonical `meter_label` is `ZN.16-PB_child__1_ch3pm` . And `gold.energy_raw` carries no label at all.	One normalisation map, applied once, shared by Tasks 1, 3 and 4. All six meters resolve by prefix.
Usable row count	Plan §4 says "~4,260 usable rows total across 29 meters" — the leading digits of the 4,260,849 *interval* count. The true figure is 14,228 screened meter-days.	Material: the "data-sparse" argument and the conservative hyperparameters rest on a figure that is off by more than 3×.
Power BI	Plan phase 7 promotes ML tables into Power BI; the standing constraint and three documents say Power BI is untouched. Separately, the semantic model is *not* Gold-only — it runs a native query against `silver.fact_energy_readings_snapshot` — and has 10 report pages, not the documented 8.	Out of scope for this engagement. Flagging it so §9 of the pipeline doc gets corrected.
Artifact names	`label_replay_spec.md` vs `replay_spec.md` ; `run_metadata.json` vs `run_metadata_v2.json` ; three names for the output root (`outputs/` , `ml_outputs_v2/` , `model_outputs/`). Six deliverables are invoked by name but absent from the §0 tree, including `run_replay_batch.py` and `build_review_queue.py` — the only scripts phases 0 and 4 actually run.	One naming pass over the whole spec. Each mismatch fails a DoD checkbox that someone will have to argue about.
Run order	Commands use bash `$(date +%s)` on a PowerShell-primary Windows machine; two phases told to run in parallel each mint their own timestamp directory, and downstream phases take hand-typed `<task0_ts>` placeholders.	One run id minted once and passed down, so the run graph is reproducible.

What the adversarial pass overturned

Recording these because they were plausible and are wrong — and because one is a correction to something I said earlier in this review.

- **The recovered SQL is not authoritative.** I initially treated `DATABASE/05_recovered_history/original_ddl/table__silver__fact_energy_consumption_validated.sql` as the real §6.7 implementation. It is a *cancelled draft* recovered from server logs — its own header says to confirm it against the catalog, pipeline §17 excludes draft SQL from the pipeline, and the live build does not implement three of its branches (zero rows for `STARTUP_BASELINE` , `HIGH_SPIKE_RECOVERY` and `UNRESOLVED_DECREMENT` ; `meter_consumption_status` is not even a live column). **The live table is authoritative**; the recovered SQL is corroborating evidence of intent only.
- **The meter-level quarantine is not a leakage vector.** It appears only in that draft, has zero rows, and is not implemented live. Worth one sentence in the spec noting it is out of scope and dormant — not the critical finding it first looked like.
- **The startup baselines are correctly documented.** 46.5 and 150.4 kWh are consequences of timestamp predicates, and §6.7 already words them that way. My earlier note that the docs treat them as logic was wrong.
- **Instructions V2 §0.2 correctly identifies the forward-looking rules** as steps 4–5. The docs get this right.
- **Per-phase active power is the right grain** for Model 1; the omission of `active_power_total_kw` matters for Model 3's negative-TkW count, not for Model 1's detectors.
- **"Highest-leverage" is defensible as a dependency claim.** Plan §3.0 means Tasks 2 and 3 are blocked on Task 0, and in that sense it is true. What does not survive is the implied *metric impact* — that is 0.00%, per Finding A.

Section 06

Handling the missing Python files

I searched the entire user profile, not just the project. **All thirteen historical artifacts are absent** — `02_prepare_daily_data.py` , `03_ml_experiment.py` , `test_ml_experiment.py` , `daily_consumption_profile.csv` , `meter_ml_summary.csv` , `requirements_ml_snapshot.txt` , `README_ML_NEXT_STEPS.md` , the four result CSVs,

`run_notes.json` and the whole `ml_outputs/` tree. The only Python in the project is `DATABASE/export_database.py`, `DATABASE/verify_recovered_sql.py` and `LIVE_DASHBOARD/serve.py`.

So every instruction of the form "port it as-is", "reuse the existing screen logic unchanged" and "reuse the existing unit tests" is unsatisfiable. I will not claim to have ported or reused anything. Each reference gets reconstructed from the authoritative current-state document and labelled as a reconstruction:

DOCUMENT REFERENCE	RECONSTRUCTED FROM	CONFIDENCE
Quality screen in `02_prepare_daily_data.py`	Pipeline §11.2 — all six conditions are stated, and the resulting counts (16,704 / 1,951 / 58 / 2,421 / 14,228) give a reconciliation target to hit	`HIGH — VERIFIABLE`
Interval→day assignment	Pipeline §11.2 plus the live `reading_date_saudi` column; total must land on 2,020,697.5 kWh and 14,753 meter-days	`HIGH — EXACT CHECK`
Feature set in `03_ml_experiment.py`	Pipeline §11.4 — the seven features are named, though the annual encoding is described only as "annual calendar position"	`MEDIUM`
Evaluation windows and minimums	Pipeline §11.4 — exact dates and the 120/20 minimums	`HIGH`
The eight tests in `test_ml_experiment.py`	Pipeline §11.6 lists all eight by name; each is restatable as an assertion	`MEDIUM — NAMES ONLY`
Read-only connection pattern	**Real working code** in `DATABASE/export_database.py` — env-var-or-getpass password, libpq connect options, post-connect verification	`HIGH — RUNS TODAY`
The §6.7 cleaning rule	The **live table**, cross-read against pipeline §6.7 and the recovered draft as corroboration only	`HIGH — 4.26M ROWS TO CHECK AGAINST`

Two pieces cannot be fully recovered and need a decision rather than a guess: **how duration coverage is computed** for the 95–100% screen (which denominator, and how partial intervals at Saudi midnight are apportioned), and **how boundary days are identified** per meter. Both are stated as outcomes in §11.2 but not as algorithms. I will implement the reading that reproduces the published counts exactly, and record the assumption in the spec — the 58 boundary days resolving to 29 meters × 2 is a strong check that the reading is right.

> RULE FOR THE REBUILD
>
> Where a reconstruction cannot be pinned by a published number, it gets written down as an explicit assumption in `replay_spec.md` and carried into `run_metadata` — never silently chosen to make a result look better. No existing data-quality or electrical business rule will be changed to improve an ML metric.

Section 07

Proposed `ML_WORK/` structure

Close to Instructions V2 §0, with the gaps filled: the scripts the run order invokes but the tree omits, a single environment, a pinned requirements file, and every output path anchored under `ML_WORK/` rather than left relative.

```
ML_WORK/
├ .venv/                          one environment, psycopg + lightgbm together
├ requirements.txt                pinned; currently missing entirely
├ README.md                       how to run, and the safety boundary
├ config/
│  ├ db_config.yaml                host/port/db/user — never the password
│  ├ model1_config.yaml            ewma_lambda, ewma_k, cusum_k, cusum_h,
│  │                               baseline_window_days, min_load_gate, exclude_windows
│  ├ model2_config.yaml            hyperparams, window dates, margin_pp
│  └ model3_config.yaml            window_days, contamination, random_state
├ src/
│  ├ db.py                         read-only connection, libpq options, verified
│  ├ meters.py                     label normalisation + CB ratings (shared by 1/2/3/4)
│  ├ saudi_time.py                 UTC→Asia/Riyadh, day boundaries, as_of cuts
│  ├ labels/
│  │  ├ replay_spec.md             the signed-off rule statement
│  │  ├ replay.py                  replay_validated_kwh()
│  │  ├ daily_profile.py           reconstructed §11.2 screen + daily rollup
│  │  └ run_replay_batch.py        driver — absent from the §0 tree
│  ├ model1_anomaly_raw/
│  │  ├ features.py  detectors.py  run_model1.py
│  ├ model2_forecaster/
│  │  ├ features.py  train.py  backtest.py  run_model2.py
│  ├ model3_triage/
│  │  ├ features.py  fit.py  run_model3.py
│  └ common/
│     ├ metrics.py                WAPE, MAE, quantile coverage — formulas stated
│     ├ io.py                     versioned writer + run_metadata.json
│     └ build_review_queue.py     Task 4 — absent from the §0 tree
├ tests/
│  ├ conftest.py                   fixtures — absent, and §1.4 needs one
│  ├ test_replay.py       test_daily_profile.py
│  ├ test_model1.py       test_model2_leakage.py    test_model3.py
└ outputs/
   └ run_<YYYYMMDD>T<HHMMSS>Z/     one run id minted once, passed to every phase
      ├ run_metadata.json          single name — resolves the v2 conflict
      └ *.csv / *.parquet
```

Three deliberate changes. `ml_outputs_v2/` is dropped — the promotion precedent it says it mirrors (`ml_outputs/experiment_20260907_v1`) does not exist on disk, so there is nothing to mirror; a single timestamped `outputs/` with a promotion marker in `run_metadata.json` does the same job without a second tree. `meters.py` and `saudi_time.py` are new because the label map and the UTC+3 conversion are each needed by three separate tasks and getting them inconsistent is the most likely silent bug. And `daily_profile.py` is split out of the replay, because the §11.2 screen is a reconstruction that deserves its own tests independent of the replay logic.

Section 08

Files Task 0 would create

Listed for approval — **none of this is written yet.**

PATH	PURPOSE	NOTES
`src/labels/replay_spec.md`	The rule statement, transcribed from the live table and pipeline §6.7, for sign-off before any code runs	Names the two forward-looking rules, the dormant quarantine, and every reconstruction assumption
`src/db.py`	Read-only connection with libpq options and post-connect verification	Shared by Tasks 1–3; modelled on `DATABASE/export_database.py`
`src/saudi_time.py`	UTC↔Asia/Riyadh, day boundaries, `as_of` cut construction	Shared; UTC+3 fixed, no DST
`src/meters.py`	Label normalisation and CB ratings	Shared; resolves the `ZN.16-PB` vs canonical-label mismatch once

PATH	PURPOSE	NOTES
src/labels/replay.py	`replay_validated_kwh()` — strictly backward-looking reconstruction	Per-meter streaming on the existing index
src/labels/daily_profile.py	Reconstructed §11.2 screen and Saudi-day rollup	Must reproduce 16,704 / 1,951 / 58 / 2,421 / 14,228 and 2,020,697.5 kWh
src/labels/run_replay_batch.py	Driver producing the profile and metadata	Incremental, not the quadratic re-replay the doc's pseudocode implies
src/common/io.py	Versioned writer and `run_metadata.json`	Records rule version, row counts, divergence %, package versions
tests/conftest.py	Shared fixtures	Absent from §0 and required by the Task 1 gate
tests/test_replay.py	Equivalence and no-future-rows tests	See the note below — the doc's two tests miss the point
tests/test_daily_profile.py	Screen reconstruction against published counts	The real check on the reconstruction
config/db_config.yaml	Connection parameters	**No password.** Password from `SADINHUB_PASSWORD` or prompt
requirements.txt	Pinned dependencies	Resolves the split-interpreter blocker
outputs/run_<ts>/…	`daily_consumption_profile_v2.csv`, `run_metadata.json`	Under `ML_WORK/`, as required

ONE CHANGE TO TASK 0'S TESTS

Instructions V2 §0.3 specifies two tests: that the replay matches Gold on days *not* touched by a correction, and that no future row is read. Both pass trivially without exercising the thing Task 0 exists for. Because all 23 affected readings resolve within their own day, the strong test is available and much better: assert the replay matches Gold on **all** 4,260,849 rows including the 23, then add a synthetic case placing a spike at a day's final reading — the one situation that genuinely cannot be resolved — and assert the documented carry-forward behaviour. Note also that the equality test must compare `validated_kwh_consumption` and `interval_quality`, not `consumption_cleaning_status`, which differs between the live table and the recovered draft on 3 rows.

Section 09

Decisions needed before Task 0 starts

Six of these block; the rest can be settled while Task 0 runs.

#	DECISION	WHY IT BLOCKS	MY RECOMMENDATION
1	Does Task 0 gate Tasks 2 and 3?	Sets the whole schedule	**No.** Build it in parallel, prove equivalence, swap inputs. Evidence in Finding A.
2	Forecast issue time and horizon	Determines the lag convention; worth more than the gate margin	Issue at end of Saudi day D for day D+1; `lag_1` = day D.
3	Where the promotion gate is evaluated	Decides whether the final number means anything	Gate on validation; report final test once, untouched.
4	One Python environment	Nothing runs until this is fixed	Single `ML_WORK/.venv`, pinned, with psycopg + lightgbm + pyarrow + openpyxl.
5	Reading Silver for breaker ratings	Task 2 cannot get CB rating from Gold, and no new DB objects may be created	Read `silver.dim_meter_cb_rating` directly, read-only, and record the boundary exception.

#	DECISION	WHY IT BLOCKS	MY RECOMMENDATION
6	The Model 1 acceptance set: 4, 5 or 6 meters	It is a pass/fail gate, and the phases contradict the Assessment Report	Rebuild from pipeline §13 + the Assessment Report; get ZN.10-PB.2 sourced or dropped.
7	The PF minimum-load gate	Listed as undecided in §13; Model 1 floods without it	Decide before Task 1, not during.
8	Definition of `severity`	Four deliverables require it; no document defines it	Ordinal from detector, duration, and status corroboration.
9	Does Model 3 depend on Task 0?	Changes the graph	No — it reads raw flag counts. Drop `--daily-profile`.
10	Power BI promotion (Plan phase 7)	Conflicts with the do-not-modify constraint	Out of scope here; revisit as a separate engagement.

Status — Architecture review only. No code written, no Task 0 implementation started, nothing written to the database. All database access was read-only over a verified connection; Bronze, Silver, Gold and Power BI are untouched.

Volumes, column names, distributions and the weekday/weekend comparison were verified against the live `sadinhub` database on 2026-09-16/17. The 125-finding catalogue behind Section 05 is retained in full and can be produced in detail for any dimension.